

```
--- djangoapi\core\serializers.py ---
```

```
from django.contrib.auth import authenticate
from django.contrib.auth.models import User

from rest_framework import serializers
from knox.models import AuthToken # Necesario para eliminar todos los tokens

from djangoapi.settings import REST_KNOX, DJANGO_KNOX_AUTOMATICALLY_REMOVE_TOKENS

class LoginViewWithKnoxSerializer(serializers.Serializer):
    username = serializers.CharField(help_text="The username")
    password = serializers.CharField(help_text="The user password",
                                     style={'input_type': 'password'},
    trim_whitespace=False)

    def validate_username(self, value):
        #validate_propiedad permite hacer validaciones adicionales sobre los campos
        #if len(value) > 10:
        #    raise serializers.ValidationError('Username mayor que 10')
        return value#hay que devolver un valor

    def validate(self, attrs): #se ejecuta con serializer.is_valid(raise_exception=True)
        #e inicializa validated_data, con los datos validados, que es un
dict
        username = attrs.get('username')
        password = attrs.get('password')
        #Authenticate prueba si las credenciales son correctas. No inicia sesión.
        #login sí que inicia la sesión, y crea las cookies necesarias, tradicionales.
        #Pero si usas Knox no es necesario iniciar sesiones tradicionales, excepto
        # para el sitio de administración
        user: User = authenticate(request=self.context.get('request'),
username=username,
                                password=password)

        #print(user)
        if not user:
            #raise es lo que coloca los errores en serializer.errors
            #y hace que is_valid devuelva False
            #no se genera una excepción si se ha llamado a is_valid con exception=False
            raise serializers.ValidationError({"error_request": "Wrong user or
password."})

        #Número de tokens actuales
        tokens_actuales = AuthToken.objects.filter(user=user)
        os = tokens_actuales.count()

        TOKEN_LIMIT_PER_USER=REST_KNOX['TOKEN_LIMIT_PER_USER']
        #Esto no va a pasar nunca ya que la vista borra el token más antiguo
        if os>TOKEN_LIMIT_PER_USER:
            if not DJANGO_KNOX_AUTOMATICALLY_REMOVE_TOKENS:
                raise serializers.ValidationError({"error_request": f"The maximum number
```

of opened sessions ({REST_KNOX['TOKEN_LIMIT_PER_USER']}) has been reached. Close a previous session before to get a new one."})

```
#Borra la sesión más antigua si se alcanza el límite máximo de sesiones
if DJANGO_KNOX_AUTOMATICALLY_REMOVE_TOKENS:
    TOKEN_LIMIT_PER_USER=REST_KNOX['TOKEN_LIMIT_PER_USER']
    if TOKEN_LIMIT_PER_USER <= os:
        token_antiguo = tokens_actuales.order_by('created').first()
        if token_antiguo:
            token_antiguo.delete()
            attrs['serializer_message']=f'You have been reached the maximum
login sessions: {TOKEN_LIMIT_PER_USER}. The first session has been deleted in order to
be able to get in now'
```

```
#Crea una sesión nueva
instance, token = AuthToken.objects.create(user)

attrs['token'] = token
attrs['token_expiry'] = instance.expiry
attrs['user'] = user
attrs['opened_sessions'] = AuthToken.objects.filter(user=user).count()
if not attrs.get('serializer_message', None):
    attrs['serializer_message']="Session created properly"
return attrs
```

```
class EmptySerializer(serializers.Serializer):
    pass
```

```
class LogoutAllUserSessionsSerializer(serializers.Serializer):
    username = serializers.CharField(help_text="The username. If the user does not
belong to the group admin, the token must be yours")
```